guildtrek

PYTHON FOR AI AND DATA ENGINEERING · DAY 5

Day 5, Interoperable

Almost no application stands alone. Its value comes from how cleanly it connects to other systems and other languages.

Treat every interface as a versioned, checked contract at the network edge, and cross into native code cheaply and safely at the code level.

What we will learn today

THE SEVEN QUALITIES — Performant · Scalable · Maintainable · Reliable · **Interoperable** · Trustworthy · Secure

TODAY interoperable. Connect the service to the world, safely.

Level 1 THE API LEVEL	Contracts, REST, pydantic and OpenAPI, GraphQL, real-time, testing, gRPC, messaging, webhooks, and the gateway.
Level 2 THE CODE LEVEL	The two-language pattern: a readable Python surface over a fast native core, via ctypes and friends.
Level 3 A SAFE BOUNDARY	Cross in a few large steps, watch marshaling, translate errors, and trace a request across services.

HOW EACH IDEA LANDS A teaching slide for the concept, then a demo that shows its code and proves it, then we build it for real on reviews-insight.

Two boundaries, stacked

The network boundary

Your program and another run as separate processes and talk by sending messages over the network. This is the level of web APIs.

The code boundary

Inside one running program, Python calls a function written in another language (C, Rust, Java) with no network in between. This is how much of the fast Python world is built.

THE THEME Every time you cross a boundary, it costs something and it can go wrong. Level 3 turns that into design rules for both.

LEVEL 1 · THE API LEVEL

An interface is a contract

Before any technology: an interface is a promise. Validate at the edge, version it, and fail clearly. And when a version is done, retire it on a timeline, never overnight.

Validate at the edge

Reject bad input with a clear error before it reaches your logic.

Version deliberately

Ship `/v2` while `/v1` keeps working. Retire `/v1` with a `Deprecation` header and a timeline, then `410 Gone`.

Fail clearly

Return a specific status code so a caller can tell a bad request from a broken server.

ENTERPRISE LENS A clear, versioned, checked contract lets two teams build on either side of it at once without breaking each other.

REST in practice

Name resources (nouns), let the verb say the action, and use status codes honestly.

```
GET    /v1/reviews          # list
POST   /v1/reviews          # create
GET    /v1/reviews/42       # read one
PUT    /v1/reviews/42       # replace
DELETE /v1/reviews/42       # remove
```

Code	When
200 / 201	ok / created
400 / 422	malformed / invalid value
401 / 403	not logged in / not allowed
404	not found
429 / 500	rate-limited / server error

TWO MORE HABITS Page and filter large collections (`?limit=&offset=`), and make creates safe to retry with an idempotency key.

Demo · pagination

DEMO `poetry run python demo_pagination.py`

The claim: a list endpoint returns one page plus the total, never the whole collection.

```python
@app.get("/v1/reviews")
def list_reviews(limit: int = 20, offset: int = 0):
    page = store[offset : offset + limit]
    return {"items": page, "total": len(store), "limit": limit, "offset": offset}
```

```
GET /v1/reviews?limit=2&offset=2  ->  items: 2, total: 5, limit: 2, offset: 2
```

WHAT IT MEANS The caller gets a page and the total, so it can walk the rest, and the server never has to send a whole table in one response.

The contract in code: pydantic and OpenAPI

Declare the request **and** the response as pydantic models. FastAPI validates the incoming body, checks what you return with `response_model`, and generates the OpenAPI spec at `/openapi.json` and `/docs`.

```python
class ReviewIn(BaseModel):                          # request contract
    product_id: str = Field(min_length=1)
    rating: int = Field(ge=1, le=5)

class ReviewAccepted(BaseModel):                    # response contract
    status: str

@app.post("/v1/reviews", status_code=201, response_model=ReviewAccepted)
def submit(review: ReviewIn) -> ReviewAccepted:
    return ReviewAccepted(status="accepted")
```

THE SPEC IS THE CONTRACT Other teams build against `/openapi.json`; you can import it into Postman or a code generator. A `rating: 11` is refused with a `422` before your code runs.

Demo · the REST contract

DEMO `poetry run python demo_rest_contract.py`

The claim: the versioned, checked contract is real, testable, and machine-readable.

```
client = TestClient(app)
client.get("/health")                                    # 200
client.post("/v1/reviews", json={"product_id": "P1", "rating": 5})   # 201
client.post("/v1/reviews", json={"product_id": "P1", "rating": 11})  # 422
"/v1/reviews" in client.get("/openapi.json").json()["paths"]         # True
```

```
health: 200        valid: 201        rating 11: 422        contract has /v1/reviews: True
```

WHAT IT MEANS Good input is accepted, bad input is refused at the edge, and the OpenAPI document lists the versioned route. The contract genuinely exists.

GraphQL in practice

The client shapes the query: one endpoint, and the caller names exactly the fields it wants. Fits when many clients each need a different slice of the same data.

```
type Review { productId: String!, rating: Int! }
query { reviews(productId: "P100") { rating } }
```

THE N+1 TRAP Ask for 100 reviews and each author's name from a separate table, and a naive resolver runs 1 query for the reviews plus 100 for the authors: 101 calls. Fix it with batching (a DataLoader), and limit query depth and cost.

Demo · N+1 versus batching

DEMO `poetry run python demo_n_plus_one.py`

The claim: naive nested lookups explode into 1+N calls; batching collapses them to 1+1.

```
# naive: 1 read for the list, then 1 lookup per review
for review in reviews:
    author = fetch_author(review.author_id)     # N calls

# batched: collect the ids, one lookup for all
authors = fetch_authors([r.author_id for r in reviews])   # 1 call
```

```
naive resolver  : 6 calls   (1 list + 5 authors)
batched loader  : 2 calls   (1 list + 1 batch)
```

WHAT IT MEANS Same result, far fewer calls. The DataLoader gathers the ids and fetches them in one round trip, which is the standard fix for N+1.

Real-time: WebSockets and SSE

REST, GraphQL, and gRPC are request-and-reply. Some features need the server to push updates as they happen. Polling every second does not scale; a persistent connection does.

Server-sent events (SSE)

One-way stream: the server keeps sending updates down an open HTTP connection. Right for live prices or a job's progress.

WebSockets

Two-way, always-open: either side can send at any time. Right for chat or a collaborative editor.

ENTERPRISE LENS A persistent connection removes wasteful polling but adds state: one open connection per client, harder to scale and load-balance. Prefer SSE when one-way push is enough.

gRPC, messaging, and webhooks

gRPC

Typed, binary calls between your services from one `.proto` schema. Fewer bytes on the wire and faster parsing than JSON. Not browser-friendly.

Messaging

A **queue** carries a command, consumed once (RabbitMQ). An **event stream** carries a fact, read by many (Kafka).

Webhooks

A third party calls you when something happens. Verify the signature before acting; anyone who learns the URL can post to it.

```
message Review { string product_id = 1; int32 rating = 2; }
service Reviews { rpc Submit(Review) returns (Ack); }
```

THE WEBHOOK TRAP Trusting a webhook because it arrived. A real provider signs each one; verify that signature first. (Security is Day 7.)

Demo · verify a webhook signature

DEMO `poetry run python demo_webhook_signature.py`

The claim: a signature over the body lets you accept a real webhook and reject a forged one.

```
expected = hmac.new(SECRET, body, hashlib.sha256).hexdigest()
hmac.compare_digest(sent_signature, expected)   # True for the real one, False for a forgery
```

```
valid signature  : accepted
forged signature : rejected (401)
```

WHAT IT MEANS Only a sender who knows the shared secret can produce the right signature, so a forged request cannot make your system act. Compare with `compare_digest` to avoid timing leaks.

What surrounds the API: identity and the gateway

Identity: who is calling

Authentication (who) and authorization (what they may do). Carry it with an **API key**, **OAuth2 / OIDC**, or a **JWT**. Never send plain passwords or roll your own crypto.

The gateway in front

You rarely expose a Python app directly. A gateway (Nginx, Kong, cloud) handles **rate limiting**, **TLS termination**, and routing, so your service focuses on business logic.

LIGHT TOUCH Both get a full treatment on Day 7 (Secure). Here, just know they sit around every real API.

API design: dos and don’ts

REST

- Nouns for resources, verbs for actions
- Honest status codes, never 200-with-error
- Version from day one; retire on a timeline
- Validate at the edge; page large lists
- Auth and rate limiting at the gateway

GraphQL

- Design a typed schema; agree it first
- Batch nested lookups (avoid N+1)
- Limit query depth and cost
- Return typed errors, not a whole-query failure
- Use it when clients need different slices

TESTING Exercise it by hand with Postman (import `/openapi.json`) or `curl`, and automatically with a FastAPI `TestClient` in the gate.

LEVEL 2 · THE CODE LEVEL

Python calling into another language

Below the network, the other kind of interop is calling code written in another language inside the same process: an ordinary function call that crosses from Python into compiled C.

The two-language pattern

A readable Python surface over a fast native core (NumPy and most of the data stack). You write the orchestration in Python; the heavy loops run in compiled C, C++, or Rust.

How Python reaches C

- `ctypes`: call an existing shared library, standard library
- `cffi`, Cython, pybind11: friendlier or compiled
- PyO3 (Rust), JPype (Java), Python.NET

Why you rarely compile it

If NumPy is C, why does `pip install numpy` not build C? It ships as a **wheel**, a pre-built binary for your platform. You only need a compiler when you write the native part yourself.

DAY 2 CALLBACK This is the answer to CPU-bound work: push the hot loop into native code that runs fast and releases the GIL.

Demo · the ctypes boundary

DEMO `poetry run python demo_ctypes_call.py`

The claim: Python can call a C function directly, once you declare the types and marshal the data.

```
libc.strlen.argtypes = [ctypes.c_char_p]   # declare the boundary types
libc.strlen.restype = ctypes.c_size_t
libc.strlen("hello".encode("utf-8"))       # marshal str -> bytes, then cross
```

```
C strlen("hello") = 5
```

WHAT IT MEANS The argtypes/restype declarations make the crossing safe, and the string is encoded to bytes before it can enter C. The raw two-language boundary, nothing hidden.

LEVEL 3 · A SAFE BOUNDARY

Crossing costs, and can go wrong

Whether the boundary is a network API or an in-process native call, crossing it is neither free nor automatically safe. A handful of design rules decide whether it holds up.

Coarse-grained, and the native win

Every crossing has a fixed cost. Make **few, large calls**, not many small ones. And when data crosses, it must be **marshaled** (packed and rebuilt), so design each crossing to do a lot of work for the data it moves.

DEMO `poetry run python demo_native_speed.py`

```
pure-Python CRC32 :  302.0 ms
native (zlib) CRC :    0.128 ms
speedup           : about 2400x faster in native C
```

WHAT IT MEANS Same algorithm, but the C version does the per-byte work at machine speed, in one coarse crossing. This is why hot loops belong in compiled code.

The GIL, memory, and errors

The GIL

Native code can release the GIL while it runs, so a threaded native call uses many cores.

Memory ownership

Decide which side allocates and which frees, and stick to it. A maintained binding handles this.

Error translation

Turn a far-side crash or a `500` into a Python exception your caller can handle.

ENTERPRISE LENS Cross in a few large steps, with clear memory ownership and deliberate error translation, and a fault on one side stays on that side.

Tracing a request across services

When A calls B calls C and something breaks, how do you find where? A **correlation id** (a `trace_id`), created at the edge and passed in a header on every call, ties one request's logs together across all the services.

DEMO `poetry run python demo_trace_id.py`

```
[trace=9f2a] edge received request
[trace=9f2a] service B called
[trace=9f2a] service C called
```

THE TRAP Logging in five services with no shared id, so a failed request is scattered with nothing to tie it together. Generate a `trace_id` at the edge and pass it across every boundary.

Boundary rules at a glance

Rule	Why	Example
Cross coarse-grained	Each crossing has a fixed cost	Pass the whole batch in one call
Watch the marshaling	Packing and copying can dominate	Few large payloads, not many tiny ones
Release the GIL in native code	Lets threads use many cores	A library that frees the lock during heavy work
Own the memory explicitly	A double free or leak crashes or bloats	Decide which side allocates and frees
Translate errors at the edge	A far-side failure must become local	Turn a crash or a `500` into a Python exception
Pass a trace id across	Ties one request's logs together	A `trace_id` header on every downstream call

LIVE LAB · THE BUILD

reviews-insight, made interoperable

A real interface facing the network and a fast native path in the code, applying both levels at once.

Aspect 1 · a typed API with request and response models

```python
class ReviewIn(BaseModel):
    product_id: str = Field(min_length=1)
    rating: int = Field(ge=1, le=5)

class ReviewAccepted(BaseModel):
    status: str

@app.post("/v1/reviews", status_code=201, response_model=ReviewAccepted)
def submit(review: ReviewIn) -> ReviewAccepted:
    store.append({"product_id": review.product_id, "rating": str(review.rating)})
    return ReviewAccepted(status="accepted")
```

BOTH HALVES CHECKED The edge refuses bad input with a `422`, and the response is validated on the way out against `ReviewAccepted`.

Aspect 2 · pagination, a signed webhook, and a trace id

```python
@app.get("/v1/reviews", response_model=ReviewPage)          # page a large list
def list_reviews(limit: int = 20, offset: int = 0) -> ReviewPage: ...

@app.post("/v1/webhooks/rating", status_code=201)           # verify before trusting
async def webhook(request: Request) -> dict[str, str]:
    if not hmac.compare_digest(sig, expected): raise HTTPException(401)

@app.middleware("http")                                     # one id across services
async def add_trace_id(request, call_next): ...
```

THE PRACTICAL PIECES A page plus a total, a webhook we verify before acting, and a `trace_id` returned on every response.

Aspect 3 · a native fast path, and proof

Drop to the code level: a hot checksum done in pure Python versus zlib's C, called once over the whole batch.

19	422	~1000x	clean
tests pass	bad input, at the edge	native faster	mypy strict

```
poetry run reviews summary   ->   reviews : 5   products: 3   rejected: 2
poetry run reviews serve     ->   GET /health, POST /v1/reviews, GET /v1/reviews, /openapi.json, /docs
```

MEASURED, NOT CLAIMED The contract is checked at the network edge, the new endpoints are tested, and the hot path is timed against pure Python.

What we covered

The API level

- Contracts: validate, version, retire on a timeline
- REST, pydantic and OpenAPI, GraphQL and N+1
- Real-time, gRPC, messaging, webhooks
- Identity, the gateway, testing with Postman

The code level and the build

- The two-language pattern; ctypes; wheels
- Coarse crossings, marshaling, the GIL, memory, errors, tracing
- reviews-insight: typed API, pagination, signed webhook, trace id, native path

THE REVIEW LENS Is input checked at the edge and the response on the way out? Is it versioned and paginated? Are webhooks verified? For any native call, is the boundary coarse-grained, and does a trace id cross it?

guildtrek

DAY 5 · COMPLETE

A boundary is a design decision.

Treat every interface as a versioned, checked contract, and cross into native code in a few large, safe steps. Each idea taught, its code shown in a demo, then built for real and proven at both boundaries.

Next: Day 6, Trustworthy. Tests, coverage, and a gate that proves the code is good.